

Techniques défensives et offensives des applications Web · Module 4

Les technologies de sécurisation

Les briques concrètes qui protègent une application : **prouver l'identité**, **protéger les données** et **contrôler les accès**.



Objectifs pédagogiques

À la fin de ce module, vous saurez :

- Distinguer **authentification** et **autorisation**.
- Situer LDAP, Kerberos, MFA, les **passkeys** et le **SSO / OAuth 2.0 / OpenID Connect**.
- Comprendre le **hachage** et bien stocker un mot de passe.
- Distinguer chiffrement **symétrique** et **asymétrique**, et le rôle de **TLS**.
- Lire un **certificat** et la chaîne de confiance ; choisir un modèle d'**autorisation**.
- Savoir où se situe la **cryptographie post-quantique**.

Trois questions, trois familles

Qui es-tu ? → authentification. **Que peux-tu faire ?** → autorisation. **Comment protéger la donnée en transit et au repos ?** → hachage et chiffrement.

Objectif

Savoir **choisir** la bonne technologie pour chaque besoin — pas la réimplémenter soi-même (rappel du module 3 : « don't roll your own crypto »).

Six familles de technologies

1

L'authentification

Facteurs, LDAP, Kerberos, MFA et passkeys

2

L'authentification centralisée et fédérée

SSO, fédération, OAuth 2.0, OpenID Connect

3

Le hachage

SHA-2, SHA-3, bcrypt, Argon2 — et les algorithmes obsolètes (MD5, SHA-1)

4

Le chiffrement

Symétrique (AES), asymétrique (RSA, ECC), TLS, post-quantique

5

Les clés et les certificats numériques

X.509, chaîne de confiance, gestion des secrets

6

Les modèles d'autorisation

ACL, RBAC, ABAC, architecture PDP / PEP

01

L'authentification

Prouver « qui je suis » avant d'accéder à quoi que ce soit.

Dans cette partie

- Authentification $\neq$ autorisation
- Les trois facteurs
- LDAP · Kerberos
- Le MFA en pratique
- Passkeys : l'après-mot de passe

Authentification $\neq$ Autorisation

Authentification (AuthN)

« Qui es-tu ? » — prouver son identité.

Ex. : saisir son mot de passe, présenter un badge, valider un code MFA.

Autorisation (AuthZ)

« Que peux-tu faire ? » — accorder des droits.

Ex. : cet utilisateur peut lire, mais pas supprimer.

L'ordre compte

On **authentifie d'abord** (établir l'identité), on **autorise ensuite** (vérifier les droits). Être connecté ne veut pas dire avoir tous les droits — c'est la confusion à l'origine de nombreuses failles d'accès (OWASP A01).

Les trois facteurs d'authentification



Ce que je SAIS

Mot de passe, code PIN, phrase secrète



Ce que j'AI

Téléphone, token, carte à puce, clé FIDO



Ce que je SUIS

Empreinte, visage, iris (biométrie)

MFA = combiner les facteurs

L'authentification **multifacteur** exige **au moins deux facteurs de catégories différentes** (ex. mot de passe + code du téléphone). Deux mots de passe = un seul facteur, donc **pas** du MFA.

LDAP

Définition — LDAP

LDAP (Lightweight Directory Access Protocol) est un protocole d'accès à un **annuaire** hiérarchique centralisant les **identités** (utilisateurs, groupes) et leurs attributs. **Active Directory** de Microsoft en est l'implémentation la plus répandue.

Une entrée est identifiée par son **DN** (Distinguished Name), son chemin unique dans l'arbre.

```
# une identité = un chemin dans l'arbre (DN)
uid=jdupont,ou=users,dc=exemple,dc=fr

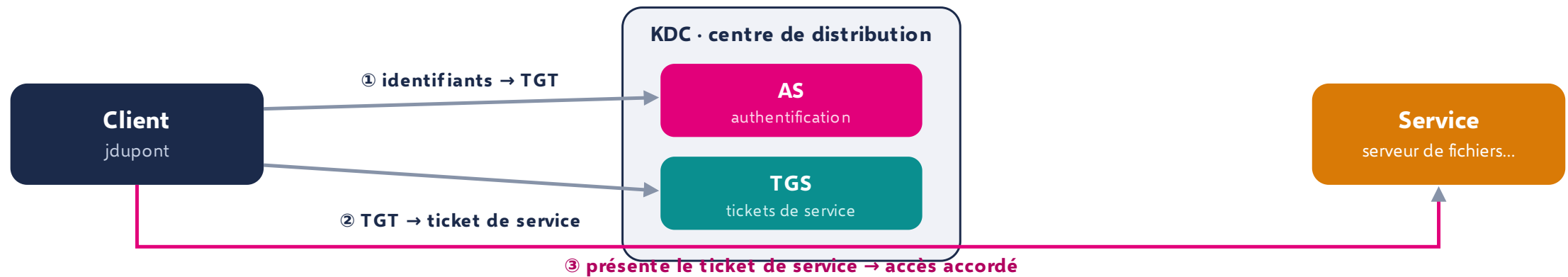
# interroger l'annuaire
ldapsearch -x -b "dc=exemple,dc=fr" \
  "(uid=jdupont)" cn mail
```

L'application délègue la vérification des identifiants à l'annuaire central.

Kerberos

Définition — Kerberos

Kerberos évite de renvoyer son mot de passe à chaque service : après une authentification unique auprès du **KDC** (centre de distribution de clés), le client obtient des **tickets** chiffrés qui prouvent son identité aux services.



Le MFA en pratique

Définition — MFA

Le **MFA** combine plusieurs facteurs. Le plus courant : mot de passe + code temporaire à 6 chiffres (**TOTP**) généré par une application (Google Authenticator, FreeOTP...).

TOTP (Time-based One-Time Password) : un code qui change toutes les 30 secondes.

Attention

Le MFA par **SMS** est vulnérable (SIM swapping, interception). Préférer une **appli TOTP** ou, mieux, une **clé FIDO** résistante au phishing.

```
# le serveur partage un secret via un QR code
otpauth://totp/MonApp:jdupont
?secret=JBSWY3DPEHPK3XP
&issuer=MonApp&period=30

# l'appli et le serveur calculent le même
# code à 6 chiffres → 492 810
```

Le secret ne circule qu'une fois (QR) ; ensuite, seuls les codes transitent.

Passkeys, FIDO2 et WebAuthn

Définition

Une **passkey** est une paire de clés (standard **FIDO2 / WebAuthn**) : la **clé privée** reste dans l'appareil (téléphone, clé de sécurité), seule la **clé publique** est chez le service. On s'authentifie en signant un défi — **aucun secret partagé ne circule**.

Concrètement : on déverrouille par biométrie ou code de l'appareil, et c'est l'appareil qui prouve l'identité.

Pourquoi c'est un progrès

- **Résistant au phishing** : la clé est liée au domaine, un faux site ne peut pas la réutiliser.
- **Rien à voler côté serveur** : pas de mot de passe haché à dérober.
- Ni ressaisie, ni base de mots de passe à protéger.

Adoption

Google, Apple, Microsoft et de nombreux services les proposent depuis 2023-2024. C'est la direction prise pour remplacer le couple mot de passe + OTP.

Les briques d'authentification

Technologie	Rôle	Où on la rencontre
LDAP	Annuaire central d'identités	Entreprise (Active Directory), intranet
Kerberos	Authentification par tickets, sans renvoyer le mot de passe	Réseaux Windows, environnements Unix
MFA / TOTP	Plusieurs facteurs pour renforcer la preuve d'identité	Partout (banque, messagerie, admin)
Passkey (FIDO2)	Authentification par clé, résistante au phishing	Grands services grand public et pro

À retenir

Ces mécanismes répondent à « **qui es-tu ?** ». Reste à gérer l'identité **entre plusieurs applications** — c'est l'objet du SSO et de la fédération.

02

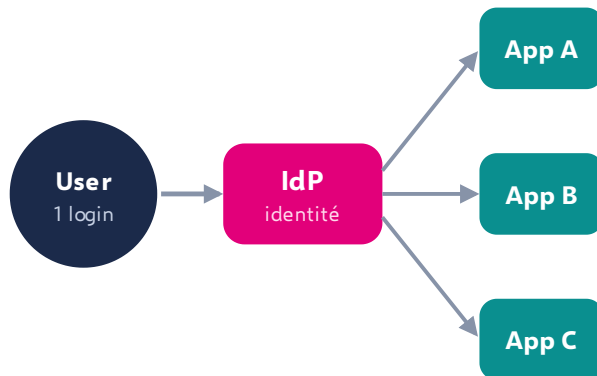
L'authentification centralisée et fédérée

Une seule identité pour de nombreux services.

Dans cette partie

- Le SSO (Single Sign-On)
- La fédération d'identité
- OAuth 2.0
- OpenID Connect
- OAuth 2.0 vs OIDC

Le SSO (Single Sign-On)



Définition — SSO

Le **Single Sign-On** permet de s'authentifier **une seule fois** auprès d'un **fournisseur d'identité (IdP)**, puis d'accéder à plusieurs applications **sans se reconnecter**.

Bénéfices : moins de mots de passe, meilleure expérience, gestion centralisée
— **désactiver un compte = couper l'accès partout**, immédiatement.

La fédération d'identité

Définition

La **fédération** permet à une application (le **fournisseur de service, SP**) de **déléguer** l'authentification à un **fournisseur d'identité (IdP)** externe, en qui elle a confiance.

C'est ce qui permet le « **Se connecter avec Google / Microsoft** » : le site ne gère pas votre mot de passe, il fait confiance à l'IdP.

Deux grands protocoles

- **SAML** — historique, très présent en entreprise (XML).
- **OpenID Connect** — moderne, basé sur OAuth 2.0 (JSON / JWT).

Le socle

La confiance repose sur des **signatures** : l'IdP signe une assertion, le SP la vérifie avec la clé publique de l'IdP.

OAuth 2.0

Définition — OAuth 2.0

Un protocole de **délégation d'autorisation** : il permet à une application d'obtenir un **accès limité** à une ressource **au nom de l'utilisateur**, **sans jamais connaître son mot de passe** (ex. « autoriser cette appli à lire votre agenda »).



Bonne pratique

Utiliser le flux **Authorization Code + PKCE**, éviter le flux « implicite » (obsolète). Les jetons doivent être de **courte durée**, avec un **refresh token** pour les renouveler.

OpenID Connect (OIDC)

Définition — OIDC

OpenID Connect est une **couche d'authentification** ajoutée par-dessus OAuth 2.0. Là où OAuth 2.0 délègue un **accès**, OIDC prouve **qui est l'utilisateur** via un **ID Token** (un JWT signé).

C'est ce qui fait fonctionner « Se connecter avec Google ».

Piège JWT

Toujours **vérifier la signature** et l'émetteur (`iss`), refuser `alg:none`, contrôler l'expiration (`exp`) et l'audience (`aud`).

```
// ID Token (JWT) décodé – le payload
{
  "iss": "https://accounts.google.com",
  "sub": "10769150350006150700748",
  "email": "jdupont@exemple.fr",
  "aud": "mon-app.fr",
  "exp": 1789999999
}
```

Le SP vérifie la **signature** et l'émetteur (`iss`) avant de faire confiance.

OAuth 2.0 vs OpenID Connect

	OAuth 2.0	OpenID Connect
Répond à	« Cette appli a-t-elle le droit d'accéder à... ? » (autorisation)	« Qui est l'utilisateur ? » (authentification)
Délivre	Un access token (jeton d'accès)	En plus, un ID Token (JWT d'identité)
Usage type	« Autoriser à lire mon agenda »	« Se connecter avec Google »

Confusion fréquente

OAuth 2.0 **seul** n'est **pas** un mécanisme d'authentification. Pour « se connecter avec... », c'est **OIDC** qu'il faut. Utiliser OAuth 2.0 pour authentifier est une erreur de conception courante.

03

Le hachage

Une empreinte à sens unique, au cœur de l'intégrité et des mots de passe.

Dans cette partie

- La fonction de hachage
- Hachage $\neq$ chiffrement
- Bien stocker un mot de passe
- Le bon algorithme par usage
- Les algorithmes obsolètes

Qu'est-ce qu'une fonction de hachage ?

Définition

Une **fonction de hachage** transforme une donnée de taille **quelconque** en une empreinte de taille **fixe**. Elle est **à sens unique** (irréversible) et **déterministe** : la même entrée donne toujours la même empreinte.

"bonjour"

#

2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

"bonjouR"

#

alc2f9...07 – empreinte totalement différente (effet d'avalanche)

Effet d'avalanche

Une seule lettre change → l'empreinte change **entièrement**.
Impossible de deviner l'entrée à partir de la sortie.

Propriété clé

Résistance aux **collisions** : il doit être infaisable de trouver deux entrées différentes donnant la même empreinte. C'est ce qui garantit l'intégrité.

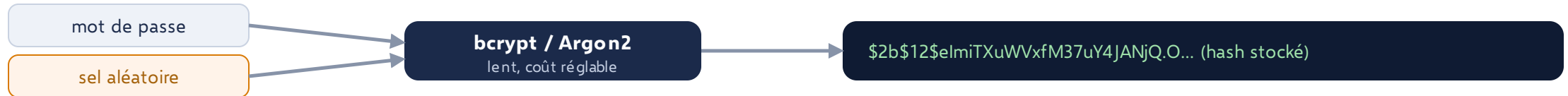
Hachage $\neq$ chiffrement

	Hachage	Chiffrement
Réversible ?	Non — à sens unique	Oui — avec la clé, on retrouve la donnée
Sortie	Taille fixe (empreinte)	Taille proche de celle de l'entrée
Clé ?	Aucune (mais un « sel » pour les mots de passe)	Oui, une clé secrète ou une paire de clés
À quoi ça sert	Vérifier l'intégrité, stocker un mot de passe	Garder une donnée confidentielle

Erreur classique

« Chiffrer » un mot de passe. On ne **chiffre pas** un mot de passe (réversible = récupérable en clair par qui a la clé) — on le **hache** (avec sel et algorithme lent). Un mot de passe ne doit jamais pouvoir être « déchiffré ».

Sel + hachage lent



Le sel

Une valeur **aléatoire unique par utilisateur**, stockée avec le hash. Il empêche les **tables précalculées** (rainbow tables) et rend deux mots de passe identiques... différents en base.

Le hachage lent

bcrypt / Argon2 sont **volontairement coûteux** (temps, mémoire) pour rendre le brute-force très lent. Le coût est réglable pour suivre la puissance du matériel.

Intégrité vs mots de passe



Intégrité / signatures

On veut un hachage **rapide** : **SHA-2** (SHA-256) ou **SHA-3**.

```
echo -n "contrat.pdf" | sha256sum
```



Mots de passe

On veut un hachage **lent et salé** : **Argon2** (recommandé), **bcrypt**, **scrypt**.

```
# Argon2 : coût mémoire + temps réglables  
$argon2id$v=19$m=65536,t=3,p=4$...
```

La règle à ne jamais oublier

Ne jamais utiliser SHA-256 **seul** pour un mot de passe : il est trop rapide → des milliards d'essais par seconde deviennent possibles sur GPU.
Rapide = bien pour l'intégrité, catastrophique pour les mots de passe.

Les algorithmes obsolètes : MD5, SHA-1

Le problème — collisions

Une **collision**, c'est deux entrées **différentes** qui produisent la **même empreinte**. **MD5** et **SHA-1** sont aujourd'hui **cassés** : on sait fabriquer des collisions à volonté.

Conséquence : on peut forger un faux document ayant la même empreinte qu'un vrai — l'intégrité n'est plus garantie.

Règles simples

- Intégrité / signature → **SHA-256** ou SHA-3, jamais MD5/SHA-1.
- Mots de passe → **Argon2** ou bcrypt, jamais un simple SHA.

Nuance

MD5 reste toléré pour des usages **non sécuritaires** (somme de contrôle rapide, déduplication), mais **jamais** pour la sécurité.

04

Le chiffrement

Rendre une donnée illisible pour quiconque n'a pas la clé.

Dans cette partie

- Symétrique vs asymétrique
- AES
- RSA et courbes elliptiques
- Le chiffrement hybride
- TLS, le « S » de HTTPS
- Le cap post-quantique

Symétrique vs asymétrique

Symétrique — une seule clé

La **même clé** chiffre et déchiffre. **Rapide** (AES), mais il faut **partager la clé** de façon sûre au préalable.



La **même clé K** des deux côtés — à convenir au préalable.

Asymétrique — paire de clés

Une clé **publique** (diffusable) chiffre, une clé **privée** (secrète) déchiffre. **Pas de secret à partager**, mais plus **lent**.



Deux clés d'une même paire (celle de Bob) — rien à partager.

Le compromis

Le **symétrique** est rapide mais impose de partager la clé. L'**asymétrique** règle le partage mais il est lent. D'où leur **combinaison** : le chiffrement hybride.

AES

Définition — AES

AES (Advanced Encryption Standard) est le standard de chiffrement **symétrique** par blocs. Rapide et robuste, il chiffre l'essentiel des données au repos et en transit, avec des clés de **128** ou **256** bits.

Usage : disques, bases, fichiers, et à l'intérieur de TLS.

Bon réflexe

Préférer un mode **authentifié** (AES-**GCM**) qui garantit aussi l'**intégrité**, plutôt qu'un simple CBC.

```
# chiffrer un fichier avec AES-256-GCM
openssl enc -aes-256-gcm -salt \
-in secret.txt -out secret.enc

# déchiffrer (même clé / mot de passe)
openssl enc -d -aes-256-gcm \
-in secret.enc -out secret.txt
```

RSA et courbes elliptiques (ECC)

Définition

Le chiffrement **asymétrique** utilise une **paire de clés** : une **publique** (diffusable) et une **privée** (secrète). **RSA** est historique ; les **courbes elliptiques (ECC)** offrent la même sécurité avec des clés **bien plus courtes**.

Sert aussi aux **signatures numériques** : signer avec la clé privée, vérifier avec la publique.

```
# générer une paire RSA 3072 bits
openssl genrsa -out prive.pem 3072
openssl rsa -in prive.pem -pubout \
-out publique.pem
```

```
# ECC : clés bien plus courtes, même sécurité
openssl ecparam -genkey -name prime256v1
```

Le chiffrement hybride



C'est ce que fait TLS

On utilise l'**asymétrique** (lent) uniquement pour **échanger une clé AES** en sécurité, puis l'**AES** (rapide) pour **chiffrer tout le trafic**. On combine la sécurité de l'échange et la vitesse du symétrique.

Bonus — confidentialité persistante

Avec l'échange **Diffie-Hellman éphémère** (ECDHE), une clé de session compromise plus tard ne révèle pas les échanges passés (forward secrecy).

TLS — le « S » de HTTPS

Définition — TLS

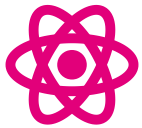
TLS (Transport Layer Security, successeur de SSL) chiffre les communications réseau. Il assure **confidentialité** (chiffrement), **intégrité** (rien n'est altéré) et **authenticité** du serveur (via son certificat).



Bonnes pratiques

TLS 1.2 minimum, idéalement **1.3** ; désactiver SSL et TLS 1.0/1.1 ; forcer HTTPS via **HSTS** ; certificat valide et non expiré.

La cryptographie post-quantique



menace le futur

RSA · ECC
vulnérables

ML-KEM · ML-DSA
résistants

Le problème

Un **ordinateur quantique** suffisamment puissant casserait **RSA** et **ECC** (mais pas AES-256 ni SHA-256, seulement affaiblis). La menace « **récolter maintenant, déchiffrer plus tard** » concerne dès aujourd'hui les données à longue durée de vie.

La réponse — standards NIST 2024

Trois standards publiés en août 2024 : **ML-KEM** (FIPS 203, échange de clés), **ML-DSA** (FIPS 204) et **SLH-DSA** (FIPS 205, signatures). La migration a commencé, souvent en mode **hybride** (classique + post-quantique).

05

Les clés et les certificats numériques

Comment prouver « cette clé publique appartient bien à ce site ».

Dans cette partie

- Le certificat X.509
- La chaîne de confiance
- Obtenir un certificat (ACME)
- La gestion des clés et des secrets

Le certificat numérique (X.509)

Définition

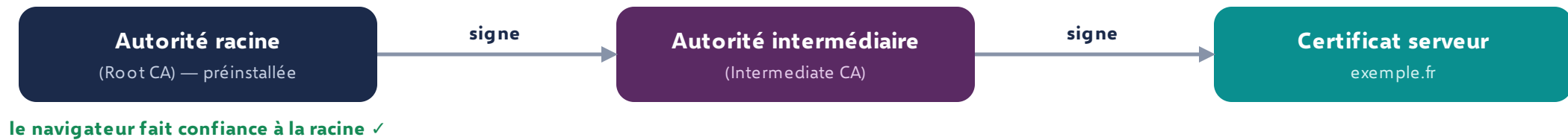
Un **certificat** (norme **X.509**) est un document **signé par une autorité de confiance** qui lie une **identité** (un domaine) à une **clé publique**. C'est ce que présente un site en HTTPS.

Il contient : le **sujet**, la **clé publique**, l'**émetteur**, les **dates de validité** et la **signature** de l'autorité.

```
# lire le contenu d'un certificat
openssl x509 -in cert.pem -text -noout

Subject:  CN = exemple.fr
Issuer:   CN = Autorité TLS R3
Validity: 2026-01-01 → 2026-04-01
Public-Key: (2048 bit RSA)
```

Qui signe qui ?



Le principe

Le navigateur ne connaît que les **autorités racines** (préinstallées dans son magasin). Si la chaîne **racine** → **intermédiaire** → **serveur** est valide et signée, le certificat du site est accepté. Un maillon invalide ou expiré → **alerte de sécurité**.

Let's Encrypt et le protocole ACME

Définition — ACME

ACME (Automatic Certificate Management Environment) est le protocole qui **automatise** la demande, la validation et le renouvellement des certificats. **Let's Encrypt** l'a rendu gratuit et universel.

On prouve qu'on contrôle le domaine (un fichier ou un enregistrement DNS), et le certificat est émis **automatiquement**, puis renouvelé sans intervention.

```
# obtenir + installer un certificat (Certbot)  
certbot --nginx -d exemple.fr
```

```
# renouvellement automatique (déjà planifié)  
certbot renew --dry-run
```

Pourquoi ça compte

Les certificats sont désormais de **courte durée** (jusqu'à ~90 jours, et la tendance est au raccourcissement). L'automatisation n'est plus optionnelle.

La gestion des clés

Une clé privée compromise annule toute la sécurité. Il faut gérer son **cycle de vie** :

- **Génération** sûre (aléatoire de qualité).
- **Stockage** protégé (coffre, HSM), jamais dans le code.
- **Rotation** régulière et **révocation** si compromise.

Le piège n°1

Les **secrets en dur** dans le code ou dans Git (clés d'API, mots de passe). C'est exactement ce que détectent les outils comme **Gitleaks** (vus au module 3).

Bonne pratique

Centraliser les secrets dans un **coffre** (HashiCorp Vault, AWS Secrets Manager...) et les **injecter à l'exécution**, jamais les committer. Un **HSM** protège les clés les plus sensibles dans du matériel dédié.

06

Les modèles d'autorisation

Une fois l'identité prouvée : qui a le droit de faire quoi ?

Dans cette partie

- Trois façons de décider des droits
- ACL
- RBAC
- ABAC et le contexte
- L'architecture PDP / PEP

Trois façons de décider des droits

Modèle	Principe	Exemple
ACL (Access Control List)	Une liste de droits par ressource	Ce fichier : Alice lit, Bob écrit
RBAC (Role-Based)	Droits attribués via des rôles	Rôle « comptable » → accès à la facturation
ABAC (Attribute-Based)	Droits selon des attributs et le contexte	« Accès si service = RH ET heure ouvrée »

En pratique

Le **RBAC** est le plus répandu (simple et lisible) ; l'**ABAC** gère les cas fins et contextuels ; les **ACL** conviennent aux droits par ressource (fichiers, objets). On les combine souvent : des rôles enrichis d'attributs.

ACL — Access Control List

Définition

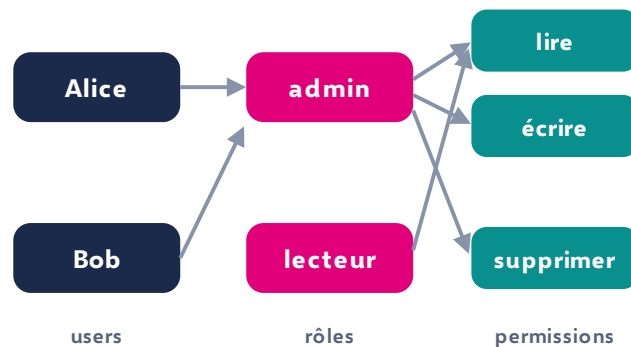
Une **ACL** attache à **chaque ressource** la liste des sujets autorisés et de leurs droits (lire, écrire, exécuter). C'est le modèle des **systèmes de fichiers**.

Simple et précis, mais **lourd à gérer** quand il y a beaucoup d'utilisateurs et de ressources : les droits se dispersent.

```
# droits d'un fichier (style Unix)
-rw-r-----  alice  compta  budget.xlsx

# ACL détaillée
user:alice:rw-
user:bob:r--
group:compta:r--
```

RBAC — Role-Based Access Control



Définition — RBAC

Les permissions ne sont pas données aux utilisateurs **directement**, mais à des **rôles** ; on assigne ensuite des rôles aux utilisateurs. Gérer les droits revient à gérer quelques **rôles**, pas des milliers d'utilisateurs.

```
// Spring Security (module 3)
@PreAuthorize("hasRole('ADMIN')")
public void supprimer() { ... }
```

ABAC — Attribute-Based Access Control

Définition — ABAC

L'**ABAC** décide en fonction d'**attributs** : ceux du **sujet** (service, niveau), de la **ressource** (sensibilité), de l'**action** et du **contexte** (heure, lieu, appareil).

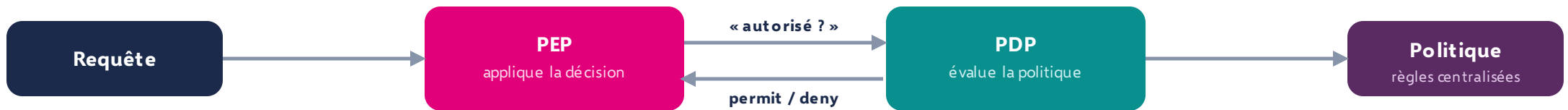
Bien plus **fin et dynamique** que le RBAC, mais aussi plus **complexe** à écrire et à auditer.

```
# politique ABAC (pseudo-règle)
permit if
  sujet.service == "RH" and
  ressource.type == "paie" and
  contexte.heure in 8..19 and
  contexte.reseau == "interne"
```

Tendance

C'est la base du **Zero Trust** : chaque accès est évalué selon le contexte, jamais accordé une fois pour toutes.

L'architecture PDP / PEP



Séparer décision et application

Le **PEP** (Policy Enforcement Point) **applique** la décision au niveau de la ressource ; le **PDP** (Policy Decision Point) la **calcule** à partir d'une **politique** centralisée. Découpler les deux rend les règles cohérentes et faciles à faire évoluer.

Les sigles du module en un coup d'œil

AuthN / AuthZ — Authentification / Autorisation

KDC / AS / TGS — infrastructure Kerberos

TOTP — mot de passe à usage unique temporel

SSO — Single Sign-On (connexion unique)

OAuth 2.0 — délégation d'autorisation

PKCE — protection du flux OAuth

SHA-2 / SHA-3 — hachage rapide (intégrité)

RSA / ECC — chiffrement asymétrique

PFS — confidentialité persistante (forward secrecy)

HSM — coffre matériel à clés

RBAC / ABAC — contrôle d'accès par Rôle / par Attribut

LDAP — protocole d'annuaire d'identités

TGT — Ticket-Granting Ticket

FIDO2 / WebAuthn — standard des passkeys

IdP / SP — fournisseur d'Identité / de Service

OIDC — OpenID Connect (identité sur OAuth 2.0)

Sel (salt) — aléa unique ajouté avant de hacher

Collision — deux entrées, même empreinte

TLS / SSL — sécurité du transport (le S de HTTPS)

X.509 / CA — certificat / autorité de certification

PQC · ML-KEM · ML-DSA — cryptographie post-quantique

PDP / PEP — décision / application des politiques

DN — Distinguished Name (chemin d'une entrée)

MFA — Multi-Factor Authentication

Passkey — authentification par clé, sans mot de passe

SAML — fédération historique (XML)

JWT — JSON Web Token (jeton signé)

Argon2 / bcrypt — hachage lent pour mots de passe

AES — chiffrement symétrique standard

HSTS — force le HTTPS

ACME — automatisation des certificats (Let's Encrypt)

ACL — droits par ressource

Autres sigles à connaître (1/2)

OWASP — Open Web Application Security Project : communauté de référence en sécurité applicative (Top 10, ASVS).

FIPS — Federal Information Processing Standards : normes techniques du NIST (ex. FIPS 203/204/205, post-quantique).

MD5 — Message Digest 5 : ancien hachage, cassé (collisions) — à bannir en sécurité.

CBC — Cipher Block Chaining : ancien mode d'AES, sans authentification intégrée.

ML-DSA / SLH-DSA — signatures post-quantiques du NIST (FIPS 204, ex-Dilithium / FIPS 205).

NIST — National Institute of Standards and Technology : organisme américain qui normalise la cryptographie (AES, SHA, post-quantique).

RFC — Request For Comments : les spécifications techniques d'Internet (OAuth, OIDC, TLS...).

GCM — Galois/Counter Mode : mode de chiffrement authentifié d'AES (confidentialité + intégrité).

ECDHE — Elliptic Curve Diffie-Hellman Ephemeral : échange de clés éphémère, base de la forward secrecy (PFS) en TLS.

ANSSI — Agence nationale de la sécurité des systèmes d'information : l'autorité française de cybersécurité.

SHA — Secure Hash Algorithm : famille de hachage (SHA-1 obsolète ; SHA-2 et SHA-3 sûrs).

AEAD — Authenticated Encryption with Associated Data : chiffrement qui garantit aussi l'intégrité (ex. AES-GCM).

ML-KEM — Module-Lattice Key Encapsulation Mechanism : échange de clés post-quantique (FIPS 203, ex-Kyber).

Autres sigles à connaître (2/2)

CA — Certificate Authority : l'autorité qui signe et garantit les certificats.

CTAP — Client to Authenticator Protocol : dialogue entre le navigateur et la clé FIDO (FIDO2 = WebAuthn + CTAP).

HTTP / HTTPS — HyperText Transfer Protocol (Secure) : le protocole du Web, chiffré par TLS en HTTPS.

API — Application Programming Interface : interface d'accès programmatique à un service.

CN — Common Name : le domaine inscrit dans un certificat X.509.

OTP — One-Time Password : mot de passe à usage unique (TOTP en est la version temporelle).

SPA — Single Page Application : application web monopage (React, Vue...), concernée par PKCE.

OPA — Open Policy Agent : moteur open source de politiques d'autorisation (ABAC, rôle de PDP).

CRL / OCSP — Certificate Revocation List / Online Certificate Status Protocol : signalent un certificat révoqué.

SIM swapping — détournement d'une carte SIM : rend le second facteur par SMS vulnérable.

XML / JSON — formats d'échange de données (SAML utilise XML ; OIDC et les JWT, JSON).

AD — Active Directory : l'annuaire de Microsoft, implémentation de LDAP avec Kerberos.

Quelle technologie pour quel besoin ?

Besoin	Technologie
Prouver l'identité	LDAP, Kerberos, MFA , passkeys (FIDO2)
Une identité pour plusieurs applications	SSO , OAuth 2.0, OpenID Connect
Stocker un mot de passe	Hachage salé et lent (Argon2, bcrypt)
Vérifier l'intégrité d'un fichier	SHA-256 / SHA-3
Confidentialité des données	Chiffrement AES (+ RSA/ECC pour l'échange)
Communications réseau sûres	TLS 1.3 + certificat X.509 valide
Préparer l'ère quantique	ML-KEM / ML-DSA (standards NIST 2024)
Décider des droits	ACL, RBAC , ABAC, via PDP / PEP

Sources et références

Standards et protocoles

- **OAuth 2.0** (RFC 6749) · **OpenID Connect** · openid.net
- **FIDO2 / WebAuthn** — fidoalliance.org, w3.org
- **X.509, TLS 1.3** (RFC 8446) · **ACME** (RFC 8555)
- **NIST FIPS 203/204/205** — cryptographie post-quantique

Bonnes pratiques

- **OWASP** — Authentication, Password Storage, JWT Cheat Sheets
- **ANSSI** — recommandations cryptographiques et TLS

Outils cités

- **OpenSSL, Certbot** / Let's Encrypt
- **Keycloak** — IdP open source (SSO, OIDC, SAML)
- **HashiCorp Vault, OPA** / OpenFGA — secrets et politiques

Points à actualiser

Les **recommandations de tailles de clés** et la **durée de vie des certificats** évoluent ; la **migration post-quantique** avance vite. Vérifier les paramètres conseillés (ANSSI, NIST) avant chaque session.

Fin du module 4

Des questions ?

Nous avons les **briques** : identité, chiffrement, certificats, autorisation. Place aux **vulnérabilités concrètes** — l'OWASP Top 10, où l'on verra ce qui casse quand ces briques sont mal employées.